

PuffChain: A Dynamic Scaling Blockchain System With Optimal Effective Throughput

Jie Xu , Sen Peng , Cong Wang , *Fellow, IEEE*, and Xiaohua Jia , *Fellow, IEEE*

Abstract—Despite its promising potential, scalability issues have hindered the widespread adoption of blockchain applications. Existing blockchain systems incur high communication and computation costs, resulting in poor throughput. Furthermore, they do not provide flexible scalability to match the dynamic changes in user transaction demand. In this paper, we present PuffChain, a dynamic scaling blockchain system that can dynamically scale up the transaction throughput when there is a surge of user demand while automatically scaling down upon the drop of user demand. We decouple the functionalities of blockchain nodes into three roles: packers that pack transactions into blocks, proposers that propose blocks for consensus, and validators that validate blocks. The number of packers is dynamically adjusted according to user transaction demand, achieving dynamic scalability. Proposers and validators use a three-phase consensus protocol with filters to maximize appending valid transactions while discarding invalid ones. By decoupling these functions, PuffChain enables adaptive scaling and optimized consensus efficiency. We also develop a theoretical model to analyze PuffChain’s throughput and derive the optimal parameters to achieve maximal effective throughput. Extensive experiments on Amazon EC2 clusters demonstrate that PuffChain achieves a throughput of 6061 transactions per second with 100 nodes, highlighting its potential for real-world applications.

Index Terms—Proof of Stake (PoS), dynamic scalability, consensus protocol, optimal throughput.

I. INTRODUCTION

BITCOIN [1] represents a significant advancement in decentralized electronic payment systems, leading to the rise of blockchain technology. Despite its pioneering role, Bitcoin and similar systems still face critical challenges that hinder widespread adoption, including high resource consumption due to Proof of Work (PoW) [2], limited throughput, long confirmation delays [3], [4], [5], and an inability to adapt throughput to dynamic user transaction demand [6].

Existing research aims to enhance blockchain performance but still has limitations. Some solutions, such as Algorand [7] and Ouroboros [8], improve energy efficiency via Proof of Stake (PoS) consensus, yet achieve only modest throughput gains. Other approaches, such as Conflux [9] and Prism [10], boost

throughput by concurrently appending blocks, but introduce duplicate transactions across blocks, wasting resources [11]. Moreover, these systems cannot align block generation rates with real-time user transaction demand. Even in situations with low user transaction demand, these solutions continue generating excessive blocks with no adjustment, wasting system resources [6].

Three primary challenges need to be addressed to overcome these limitations. First, serialized block generation limits the maximum achievable throughput. Many protocols restrict each round to only one proposed block, thereby imposing a ceiling on the system’s overall throughput [12]. For example, in Byzantine Fault Tolerant (BFT) based PoS blockchains [13], a proposed block can be appended to the blockchain if it is delivered to and approved by validators within a bounded time [14]. Second, absent centralized coordination in permissionless networks causes inefficient transaction processing and duplication. Without centralized coordination, miners always prefer to pack transactions with higher transaction fees, leading to duplicate transactions in concurrent blocks. Such duplication reduces the effective transaction throughput and wastes system resources. Third, inconsistent views of transaction demand prevent adaptive blockchain scaling. Because transactions received by individual nodes are different, blockchain scaling decisions may differ. Some nodes may decide to expand the blockchain, while others may do the opposite. This divergence in decision-making prevents the blockchain’s throughput from matching actual transaction demand.

In response to these challenges, we propose PuffChain,¹ a dynamic BFT-based PoS blockchain system. PuffChain stands out from existing solutions with two key advantages: 1) it adapts to dynamic user transaction demand, and 2) it optimizes the blockchain’s effective throughput. When transaction demand is high, PuffChain increases the effective throughput until it approaches the capacity of the underlying network. When transaction demand is low, it decreases the throughput to save communication and storage resources.

PuffChain employs several innovative designs to surmount these challenges. To address serialized block generation, it adopts a three-role decoupling mechanism, facilitating concurrent block generation. The functionalities of participating nodes are decoupled into three roles: *packers* that pack transactions into *data blocks*, *proposers* that generate *control blocks* containing

Manuscript received 2 September 2023; revised 20 December 2023; accepted 2 February 2024. Date of publication 8 February 2024; date of current version 30 April 2024. This work was supported by Hong Kong Research Grants Council (RGC) under Grant CityU 11213920, Grant CityU 11218521, Grant R1012-21, Grant R6021-20F, Grant RFS2122-1S04, Grant C2004-21G, and Grant C1029-22G. Recommended for acceptance by Dr. Bin Xiao. (*Corresponding author: Jie Xu.*)

The authors are with the Department of Computer Science, City University of Hong Kong, Hong Kong (e-mail: jiexu49-c@my.cityu.edu.hk; senpeng.cs@my.cityu.edu.hk; congwang@cityu.edu.hk; csjia@cityu.edu.hk).

Digital Object Identifier 10.1109/TNSE.2024.3363880

¹“PuffChain” refers to its dynamic ability to expand or shrink, akin to a pufferfish.

the pointers of data blocks, and *validators* that validate both control blocks and data blocks. Each round involves a single proposer who generates a control block, while multiple packers concurrently generate data blocks linked to the control block. The three-role decoupling allows concurrent data block generation, increasing the system's overall transaction processing capacity.

To avoid duplicate transaction packing and maximize the effective throughput, PuffChain allocates transactions to packers according to their identities (IDs). This strategy enables packers to concurrently generate data blocks without duplicate transactions. Additionally, PuffChain adopts a three-phase BFT consensus protocol embedded with filters. This approach allows valid data blocks to be appended to the blockchain while invalid data blocks are effectively filtered and discarded.

To ensure consistent and adaptive blockchain scaling, PuffChain uses the average block size appended to the blockchain in the previous round as an indicator of transaction demand. If the average block size is close to the maximum block size, it indicates high transaction demand. Conversely, if the average size is small, it indicates low transaction demand. PuffChain fine-tunes its throughput dynamically by adjusting the number of packers per round to meet the current transaction demand.

The contributions of this paper are summarized as follows:

- We propose PuffChain, a dynamic BFT-based PoS blockchain system that adapts its throughput to match dynamic user transaction demand. When transaction demand is high, PuffChain increases throughput until it approaches network capacity. When demand is low, throughput decreases to save resources.
- We propose a three-role decoupling mechanism and an innovative transaction packing method to maximize effective throughput. One proposer proposes a control block, while multiple packers generate data blocks in parallel that link to the control block without duplication, increasing the effective throughput.
- We design a three-phase BFT consensus protocol with filters to append blocks to the blockchain. The filter discards invalid blocks to maximize resource utilization. We also develop a theoretical model to derive optimal protocol parameters for maximizing effective throughput.

II. MODELS AND PROBLEM STATEMENT

A. System Model

PuffChain is a permissionless PoS blockchain, allowing nodes with stakes to participate in the consensus protocol. Each participating node i has a stake weight w_i , reflecting its number of stakes, along with a public-secret key pair (pk_i, sk_i) . Nodes are assigned to specific roles via a role sortition algorithm. The roles include:

- **Packers:** Packers are responsible for packing transactions into data blocks. The number of packers is dynamically adjusted according to transaction demand.

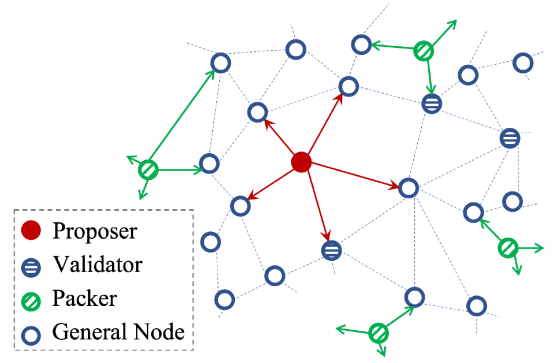


Fig. 1. Message flow of the underlying network.

- **Proposers:** One node is selected as the proposer each round to generate a control block that links data blocks and appends them to the blockchain.
- **Validators:** The validators run a three-phase consensus protocol with filters to validate the control block and linked data blocks.

Nodes not selected for these roles still remain part of the network, contributing to its robustness and security, but do not actively participate in the block generation or validation processes during that round.

B. Network Model

PuffChain adopts the same network assumptions as the previous consensus protocols [13], [15], [16], [17], which is also implied in Bitcoin [1]. First, all honest nodes are connected to a peer-to-peer network, as shown in Fig. 1. Nodes communicate with each other via the gossip protocol. Honest nodes broadcast new blocks as soon as they are received or generated, and they always extend the longest valid blockchain. A message and its signature pair are denoted as $\langle m \rangle_{\sigma_i}$, where m represents the message and σ_i signifies the signature of the digest of m from node i . Second, communication between honest nodes is synchronous [15], [16], with known and bounded message delivery time. If an honest node sends a message at time t_0 , all other honest nodes receive it within $t_0 + \Delta$.

Previous research shows that the network latency is proportional to the diameter of the network [7]. Assume that the underlying network has a diameter of h , meaning that a message sent by any node can reach any other node within a maximum of h hops, the bandwidth of each link is b , and the maximum end-to-end propagation delay of the network is d . Adapting the same underlying network model as Prism [10], the network latency Δ for a message of length S is expressed as:

$$\Delta = \frac{hS}{b} + d. \quad (1)$$

C. Threat Model

The set of participating nodes is represented as $\mathcal{N}$. Some nodes are controlled by the probabilistic polynomial-time Byzantine adversary $\mathcal{A}$, and the rest are honest nodes, denoted by $\mathcal{H}$. The proportion of the stakes held by adversarial nodes to

the total system stakes is β , and $\beta < \frac{1}{3}$. The corruption model is *slowly adaptive*, which is the same as the one adopted by previous works [15], [17]. The protocol is executed in rounds. Adversarial nodes are online from round 0 [18]. The node sets $\mathcal{A}$ or $\mathcal{H}$ are determined before each round of protocol execution and remain unchanged within that round. Adversarial nodes may act arbitrarily, such as joining, leaving, retransmitting, or reordering messages, but cannot modify a message without detection. Although adversarial nodes can delay messages, messages will eventually be delivered.

D. Problem Statement

The problem of our concern is to design a blockchain system that optimizes effective throughput while maintaining security and liveness. Blockchain throughput can adapt to dynamic user transaction demand. The system should achieve high throughput during peak user transaction demand, approaching the capacity of the underlying network. Conversely, it should scale down to save resources when transaction demand is low. The system avoids duplicate transactions for maximum resource utilization.

III. DESIGN OF DECOUPLING FUNCTIONALITIES IN POS BLOCKCHAIN

A. Overview of PuffChain

PuffChain is a permissionless BFT-based PoS blockchain system. It involves two steps for consensus:

- 1) Nodes are selected based on their stake to perform their respective roles as packers, proposers, and validators. Packers pack transactions into data blocks, proposers propose control blocks that link to the data blocks, and validators validate both control blocks and data blocks.
- 2) A BFT protocol, such as PBFT [19], is implemented to achieve Byzantine Fault Tolerance, enabling consensus even in the presence of malicious or faulty nodes. The BFT consensus ensures that the proposed block is agreed upon and added to the blockchain through a series of voting and validation phases.

PuffChain proceeds in rounds, starting from the genesis block at round 0. At the beginning of each round, nodes run a role sortition algorithm to determine their assigned roles: packer, proposer, or validator. Each round involves one proposer, multiple validators, and packers. The number of packers in a round is dynamically adjusted according to the average size of data blocks in the previous round, which indicates transaction demand. Packers pack transactions into data blocks and broadcast their packer proofs along with the generated data blocks. The proposer generates a control block containing pointers to its selected data blocks, as shown in Fig. 2, and broadcasts its proposer proof and the generated control block. Validators first broadcast their validator proofs, then validate and vote on the control block and the data blocks it points to.

Then, the proposer and validators use a three-phase BFT consensus protocol with filters to append control blocks and verified data blocks to the blockchain. During this consensus process, they only broadcast their proposed whitelists of valid

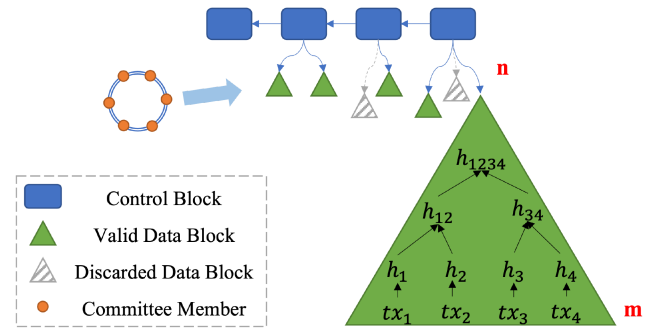


Fig. 2. Overview of PuffChain.

data blocks, thus discarding the invalid ones. Each control block points to the previous committed control block, forming a chain of blocks, as illustrated in Fig. 2. Once a node receives a committed control block, it can advance to the subsequent round.

We introduce the three-role decoupling in this section. We introduce the process of the three-phase consensus protocol with filters in the next section.

B. Three-Role Decoupling

1) *Role Sortition*: Role selection is challenging in permissionless PoS blockchains. First, role selection must ensure that the probability of a node being selected is proportional to its stake in resisting Sybil attacks. Second, role selection must be bias-resistant and unpredictable so participants cannot manipulate the selection or prepare targeted attacks in advance. Third, the selection should be efficient without relying on resource-intensive computations. Finally, the selection should be publicly verifiable to ensure accountability and transparency.

The role sortition algorithm in PuffChain addresses these challenges by using Verifiable Random Functions (VRF) [20] and cryptographic sortition to achieve secure, fair, and unpredictable node selection. Drawing inspiration from Algorand [7], each unit of stake is treated as a separate *sub-user*. Consequently, a node with a stake weight of w_i is regarded as controlling w_i separate sub-users. If a node is selected u times, it can participate as u sub-users.

A node starts the Role Sortition Algorithm as shown in Algorithm 1 in round r if it receives a committed control block in round $r - 1$. The process consists of two steps: lottery generation and local verification.

1) *Lottery Generation*: Each node generates a VRF hash and proof for each role. The role can be either a *packer*, *proposer*, or *validator*.

Specifically, the node uses its secret key sk and the current round's seed $seed^r$ to calculate the VRF hash γ and proof π . The process is defined as follows:

$$\begin{aligned}\gamma &\leftarrow \text{VRF.Hash}(sk, r || seed^r || role), \\ \pi &\leftarrow \text{VRF.Prove}(sk, r || seed^r || role).\end{aligned}\quad (2)$$

The seed $seed^r$ is computed by the previous round's block proposer j and is included in the control block B_C^{r-1} , where $seed^r \leftarrow \text{VRF.Hash}(sk_j, seed^{r-1} || r)$. The first seed $seed^1$ for

Algorithm 1: Role Sortition.

```

1: Input: Current round  $r$ , seed  $seed^r$ , secret key  $sk$ , node
    $id_i$  weight  $w_i$ , system weights  $W$ , total number of
   selected  $\tau$ , role  $role$ 
2: Output: VRF hash  $\gamma$ , proof  $\pi$ ,  $u$  times selected
3: procedure RoleSortition( $w_i, sk, r, seed^r, \tau, W, role$ )
4:    $\gamma \leftarrow \text{VRF.Hash}(sk, r || seed^r || role)$ 
5:    $\pi \leftarrow \text{VRF.Prove}(sk, r || seed^r || role)$ 
6:    $\psi = \frac{\gamma}{2^{256}}$  //Normalized random value
7:    $p \leftarrow \frac{\tau}{W}$  //Probability of selection per stake
8:    $u \leftarrow 0$ 
9:   while  $u \leq w_i$  do
10:    if  $u = 0$  then
11:       $I^u \leftarrow [0, (1-p)^{w_i})$ 
12:    else
13:       $P_{w_i}^k \leftarrow \binom{w_i}{k} p^k (1-p)^{w_i-k}$ 
14:       $I^u \leftarrow [\sum_{k=0}^{u-1} P_{w_i}^k, \sum_{k=0}^u P_{w_i}^k)$ 
15:    end if
16:    if  $\psi \in I^u$  then //Local verification
17:       $u$  sub-users are selected
18:      break
19:    end if
20:     $u \leftarrow u + 1$ 
21:  end while
22:  if  $u > 0$  then
23:    Broadcast  $\langle role, r, id_i, \gamma, \pi, u \rangle_{\sigma_i}$ 
24:  end if
25:  return  $u, role$ 
26: end procedure

```

round 1 is included in the genesis block when PuffChain is initialized. There is no $seed^0$ because the genesis block does not need to be generated in this way. However, it is important to note that if the control block generated by a proposer j points to an invalid data block, the proposer j is considered malicious, and the seed generated by j is also considered invalid. In this case, $seed^r \leftarrow H(seed^{r-1} || r)$.

2) Local Verification. Upon generating the lottery, each node verifies whether it has been chosen as a packer, proposer, or validator.

First, each node computes a normalized random value ψ , which is derived from the VRF hash γ . Because VRF hash γ is a random 256-bit string, the value of γ is a random number ranging from 0 to $2^{256} - 1$. The normalized random value $\psi = \frac{\gamma}{2^{256}} \in [0, 1)$.

Second, a node i with stake weight w_i divides the interval $[0, 1)$ into $w_i + 1$ consecutive sub-intervals I^u using the following definitions:

$$I^u = \begin{cases} [0, (1-p)^{w_i}) & u = 0 \\ [\sum_{k=0}^{u-1} P_{w_i}^k, \sum_{k=0}^u P_{w_i}^k) & u \in \{1, 2, \dots, w_i\} \end{cases} \quad (3)$$

where

$$p = \frac{\tau}{W},$$

Algorithm 2: Role Verification.

```

1: Input: Received  $\langle role, r, id_i, \gamma, \pi, u \rangle_{\sigma_i}$ , node's weight
    $w_i$  and public key  $pk$ , total selected  $\tau$ , total weights  $W$ 
2: Output: The selected times
3: procedure VerifyNode( $pk, id_i, r, role, \gamma, \pi, u, \tau, w_i, W$ )
4:   if  $\neg \text{VRF.Verify}(\gamma, \pi, r || seed^r || role)$  or  $\neg \text{Validity}(w_i)$ 
   then
5:     return 0
6:   end if
7:    $p \leftarrow \frac{\tau}{W}, v \leftarrow 0$ 
8:   while  $v \leq w_i$  do
9:     if  $v = 0$  then
10:       $I^v \leftarrow [0, (1-p)^{w_i})$ 
11:     else
12:       $P_{w_i}^k \leftarrow \binom{w_i}{k} p^k (1-p)^{w_i-k}$ 
13:       $I^v \leftarrow [\sum_{k=0}^{v-1} P_{w_i}^k, \sum_{k=0}^v P_{w_i}^k)$ 
14:     end if
15:     if  $\frac{\gamma}{2^{256}} \in I^v$  then
16:       return  $v$ 
17:     end if
18:      $v \leftarrow v + 1$ 
19:   end while
20:   return 0
21: end procedure

```

$$P_{w_i}^k = \binom{w_i}{k} p^k (1-p)^{w_i-k}, \quad (4)$$

τ is the number of packers/proposers/validators selected in this round. W is the total stake weight of the system, and $W = \sum_{i=1} w_i$.

Third, a node checks which sub-interval I^u its normalized random value ψ falls into. If $\psi \in I^u$, the node is assigned u units of stake, allowing it to act as u sub-users. $u = 0$ means that the node was not selected to participate in this round of consensus. If $u > 0$ and $role$ is a packer, the node can generate u data blocks in this round. If $u > 0$ and $role$ is the proposer or validator, the node has u votes to vote in each phase of the three-phase BFT protocol.

Finally, selected nodes broadcast their roles and role proofs $\langle role, r, id_i, \gamma, \pi, u \rangle_{\sigma_i}$, and other nodes validate received role proofs. The verification algorithm is provided in Algorithm 2.

A comprehensive security analysis of the Role Sortition Algorithm is provided in Section VI-A. This analysis includes proof demonstrating that the probability of being selected through this algorithm is directly proportional to each participant's weight. To defend against stake grinding attacks,² nodes must use their weights in round $r - \nu$ as an input to the role sortition algorithm in round r , where ν is a security parameter.

2) Adjustment of the Number of Packers: It is challenging to adjust the number of packers to match dynamic transaction demand. First, Locally collected transactions at individual nodes

²Stake grinding attacks involve manipulating the node selection by repeatedly transferring coins between multiple accounts to increase the odds of being selected [8].

cannot accurately reflect global demand. Different nodes may make inconsistent scaling decisions based on their limited views. Additionally, frequently fluctuating the number of packers may cause throughput to oscillate, reducing the system's stability.

To address these challenges, PuffChain uses the average size of previously committed data blocks as a consistent indicator of global transaction demand. Besides, thresholds are set to avoid frequent oscillations. The number of packers is fine-grained in response to transaction demand. These combined mechanisms enable smooth and adaptive scaling of transaction throughput in PuffChain.

Threshold for Expansion/Shrink: PuffChain expands or shrinks depending on the average size of data blocks in the previous round. If the average size of data blocks in the previous round is greater than $\alpha_u \|B_D\|_{\max}$, the current transaction demand is high, so PuffChain scales up. Conversely, if the average size of data blocks in the previous round is less than $\alpha_d \|B_D\|_{\max}$, the current transaction demand is low, so PuffChain scales down. α_u and α_d are the thresholds for PuffChain to scale up and down, respectively. $\|B_D\|_{\max}$ is the maximum size of data block. These are static parameters determined through empirical data analysis and theoretical modeling, taking into account factors such as underlying network capacity, security considerations, and user experience to optimize system performance.

Fine-grained Expansion/Shrink: PuffChain adjusts the blockchain throughput by adjusting the number of packers in each round to meet the dynamic transaction demand of users. Specifically, if PuffChain needs to scale up, the number of packers τ_{pack} for the next round will be increased by ϵ_u . As the number of packers increases, the number of data blocks in the next round will increase. If PuffChain needs to scale down, the number of packers τ_{pack} for the next round will be decreased by ϵ_d . As the number of packers decreases, the number of data blocks in the next round will decrease. ϵ_u and ϵ_d are values set according to the magnitude of change in actual transaction demand.

The minimum value of τ_{pack} is 1 to ensure the liveness of the blockchain. The maximum value of τ_{pack} is limited by the physical limits of the underlying network, which is discussed in Section V.

3) *Data Block Generation Without Duplicate Txs:* To prevent transaction duplication, PuffChain allocates transactions to the packer whose ID is closest to the transaction IDs in terms of Euclidean distance. Ties from equal distances are resolved by choosing the packer with the smaller ID. When a node is selected as a packer, it generates data blocks using Algorithm 3.

Network Monitoring: Nodes keep listening to the network to generate blocks. Each node maintains a local *transaction pool* and a *data block pool*. The received new transactions and data blocks are added to the local transaction pool and data block pool, respectively, after validation. Transactions that have been packed into data blocks are removed from the local transaction pool. Data blocks that have been appended to the blockchain are removed from the local data block pool. Besides, if a node receives a control block committed in round r , the node will delete the data blocks generated in and before round $r - r'$ from its local data block pool. r' is the validity period of data

Algorithm 3: Data Block Generation Without Duplicate Txs.

Require: Own ID id_0 , secret key sk_0 , the received packers'

IDs in round r : $pid^r = [id_1^r, id_2^r, \dots, id_k^r]$

Ensure: New data block B_D^r

```

1:  $tr \leftarrow \emptyset, T_{\max} \leftarrow \text{MaxMerkleTree} \| B_D \|_{\max}$ 
2: for all  $tx \in \text{local transaction pool}$  do
3:   if  $\|tr\| \geq T_{\max}$  then
4:     break
5:   end if //Not exceed  $\|B_D\|_{\max}$ 
6:    $tid \leftarrow tx.id$ 
7:    $d_{\min} \leftarrow \text{MinEuclid} pid^r, tid$ 
8:   if  $\text{Euclid} id_0, tid < d_{\min}$  then
9:      $tr \leftarrow tr \cup \{tx\}$ 
10:  else if  $\text{Euclid} id_0, tid == d_{\min}$  then
11:    //Handle tie with secondary criteria
12:    if  $\text{IsSmallID} tx, id_0, pid^r$  then
13:       $tr \leftarrow tr \cup \{tx\}$ 
14:    end if
15:  end if
16: end for
17:  $B_D^r \leftarrow \text{CreatDataBlock} r, id_0, tr$ 

```

blocks to resist malicious packers withholding data blocks. In this paper, $r' = 1$, which means data blocks expire in the next round. Transactions in expired data blocks are released back to the local transaction pool.

Data Block Generation: A packer can only pack transactions whose transaction IDs are closest to its ID to avoid duplicate transaction packing. Specifically, packers in the same round broadcast their pack proofs first. After a packer receives all packer proofs, it computes the Euclidean distance between the IDs of packers and the transaction IDs of transactions in its transaction pool. The packer selects transactions whose IDs are closest to its ID in Euclidean distance to compute the Merkle tree of transactions. Then, the packer generates and broadcasts a data block

$$B_{D_i}^r = \langle r, id_i, tr \rangle_{\sigma_i}. \quad (5)$$

r is the current round number, id_i is the ID of the packer i , tr is a Merkle tree composed of validated transactions. r , id_i , and tr are signed by the packer i 's secret key, and σ_i is the signature.

Data Block Validation: All nodes in the network validate data blocks after receiving them. A data block is invalid if it meets any of the following conditions: its packer is not selected, it has expired, it contains transactions that do not satisfy the closest distance, or it contains transactions that duplicate or conflict with transactions already on the blockchain. Nodes add valid data blocks to their local data block pools and forward them to their peers.

IV. THE BFT CONSENSUS PROTOCOL

Traditional BFT consensus protocols use multi-phase voting to reach a consensus on appending a block to the blockchain. However, this process could be inefficient when an invalid block

is proposed. In such scenarios, honest validators do not vote and wait for the current round to expire before starting the next round. Consequently, no transactions are processed, and no block is appended to the blockchain during this round, which wastes time and system resources.

We address this inefficiency by proposing a filtering mechanism that discards invalid data blocks put forward by malicious proposers. Instead of simply approving or disapproving all data blocks selected by the proposer, validators broadcast whitelists of data blocks. Consequently, valid data blocks are appended to the blockchain, while invalid data blocks are discarded, minimizing communication waste. Although the three-phase BFT protocol with filters described below follows the design of PBFT [19], our future research will focus on extending the filtering mechanism to other more effective BFT consensus protocols, such as ByzcoinX [16] or Hotstuff [21].

A. The Three-Phase Consensus Protocol With Filters

Nodes with different roles collaborate to add blocks to the blockchain. Packers pack transactions into data blocks and broadcast them. All nodes validate and store data blocks in their local block pools after receiving them. A proposer then generates a control block containing pointers to data blocks. Validators validate the control block and data blocks. This three-phase BFT consensus protocol with filters is then used to append the proposed control block and its validated data blocks to the blockchain.

A round of BFT consensus protocol includes three phases: *Propose*, *Validate*, and *Commit*. In *Propose* phase, all participating nodes use the role sortition algorithm to confirm their roles and broadcast their role proofs. A proposer selects data blocks from its local data block pool to generate a control block and broadcast it. In *Validate* phase, validators validate and vote on the proposed control block and the data blocks it points to. In *Commit* phase, validators confirm the votes of others and reply to all participating nodes with the final consensus result.

If a node receives a committed control block of round $r - 1$, it starts a timer and enters round r . When a node enters round r , it may be the first to finish round $r - 1$. According to the network model, the network latency for *Reply* message is $\Delta_{MR} = \frac{h\|M_R\|}{b} + d$, where $\|M_R\|$ is the size of a *Reply* message. At most Δ_{MR} time later, other nodes also finish *Reply* phase in round $r - 1$, and the nodes selected as packers/proposer/validators in round r broadcast their role proofs. The network latency for role proof is $\Delta_{role} = \frac{h\|role\|}{b} + d$, where $\|role\|$ is the size of a role proof. Thus, $\Delta_{MR} + \Delta_{role}$ time after the start of round r , a node receives all role proofs in round r . Then, the node determines the proposer and the total number of votes for this round. The proposer and validators form the committee in round r . Nodes compute the total number of committee votes $\mathcal{T}$ in round r by accumulating the votes of committee members. There may be multiple nodes selected as proposers by the role sortition algorithm. To ensure only one control block is proposed, proposers will be assigned priorities based on their role proof. Node with a smaller hash value of role proof indicates higher priority. Nodes only accept the control block from the proposer with the highest

priority. If no proposer is selected, nodes wait a timeout period. The current round will be considered expired. Nodes will then start the role selection process again to select a new proposer for the next round. This ensures the liveness of the system.

The following is the process of the three-phase BFT consensus protocol with filters.

1) *Propose Phase*: At the start of this phase, all nodes verify their roles. Validators broadcast their role proofs. Proposers and packers broadcast their role proofs and generated blocks. A proposer, say i , then selects data blocks from its local data block pool to generate a control block, denoted as:

$$B_C^r = \langle r, id_i, H(\tilde{B}_C^{r-1}), seed^{r+1}, \pi_{seed}^{r+1}, L_i^r \rangle_{\sigma_i}. \quad (6)$$

r is the current round number. id_i is the ID of the proposer i , $H(\tilde{B}_C^{r-1})$ is the hash of the committed control block in round $r - 1$. $seed^{r+1}$ and π_{seed}^{r+1} are the seed and the seed proof for round $r + 1$, respectively. $L_i^r = \{H(B_{D_1}^r), H(B_{D_2}^r), \dots\}$ is a list of data block IDs selected by the proposer after validation.

2) *Validate Phase*: If a validator j receives a control block B_C^r in round r , it first validates the proposer's role proof. Then, it validates all data blocks in L_i^r . If a data block B_D has not been received, it broadcasts a request message R_B to request unreceived data blocks. If no response within $\Delta_{BD} = \frac{h(\|R_B\| + \|B_D\|)}{b} + d$, validator j regards the data block B_D as invalid. After validation, validator j assembles the IDs of valid data blocks to a *Validate* whitelist $L_j^r = \{H(B_{D_1}^r), H(B_{D_2}^r), \dots\}$. Finally, it broadcasts a *Validate* message $\langle Validate, r, id_j, H(B_C^r), L_j^r \rangle_{\sigma_j}$ to vote for data blocks in L_j^r .

3) *Commit Phase*: Committee members collect and validate *Validate* messages from others. If a member j receives more than $\frac{2}{3}\mathcal{T}$ valid votes for a whitelist L^r , it accepts the *Validate* messages for L^r and enters the *Commit* phase. It generates and broadcasts a *Commit* message $\langle Commit, r, id_j, H(B_C^r), L^r \rangle_{\sigma_j}$.

All committee members collect and validate *Commit* messages from others. If there are more than $\frac{2}{3}\mathcal{T}$ valid votes for a whitelist L^r , member j accepts the *Commit* messages for L^r and generates and broadcasts a *Reply* message $\langle Reply, r, id_j, H(B_C^r), L^r \rangle_{\sigma_j}$.

All participating nodes wait to receive *Reply* messages. If a node receives a whitelist L^r in *Reply* message that gets more than $\frac{1}{3}\mathcal{T}$ valid votes, the consensus for round r ends and the node stops its timer. The node appends the committed control block $\tilde{B}_C^r$ and data blocks $B_{D_1}^r, B_{D_2}^r, \dots$ to its blockchain, where

$$\tilde{B}_C^r = \left\{ r, id_i, H(\tilde{B}_C^{r-1}), seed^{r+1}, \pi_{seed}^{r+1}, L^r \right\}. \quad (7)$$

B. Handling of Concurrent Conflicting Transactions

It is challenging to handle concurrent conflict transactions in systems with parallel transaction processing, including PuffChain, DAG-based blockchains [9], [22], [23], and parallel blockchains [24]. First, adversarial nodes can exploit concurrency to broadcast conflicting yet valid transactions, which are packed into different blocks, causing conflicts. Second, managing concurrent conflicting transactions requires complex scheduling algorithms, leading to increased system complexity

and low resource utilization. Third, conflicts may not be immediately detectable and only surface after blocks are committed. Multiple nodes could have conflicting perspectives on the validity of the same transaction.

To address this, PuffChain implements an optimistic conflict resolution strategy with three steps. First, nodes optimistically assume that there are no conflicting transactions between concurrent data blocks in the same round and append them to the blockchain. Second, nodes sort and prioritize transactions on the blockchain. Finally, only one of the conflicting transactions with the highest priority is preserved, and the conflicting transactions with lower priority are invalid and discarded.

Transaction prioritization is based on the following criteria:

- 1) *Round-Based Priority*: Transactions are sorted and prioritized based on the round in which they were appended to PuffChain. If two transactions tx_1 and tx_2 are appended to PuffChain in rounds r_1 and r_2 , respectively, and $r_1 < r_2$, then tx_1 comes before tx_2 and has higher priority.
- 2) *Block-Based Priority*: For transactions appended to PuffChain in the same round, transactions are sorted and prioritized based on the round in which they were included in data blocks. If two transactions tx_1 and tx_2 are appended to PuffChain in the same round and included in data blocks in rounds r_1 and r_2 respectively, and $r_1 < r_2$, then tx_1 comes before tx_2 and has higher priority.
- 3) *Transaction Identity-Based Priority*: For transactions appended to the PuffChain in the same round and packed into data blocks in the same round, the transaction with a smaller ID number is given higher priority.

This strategy ensures a deterministic and unique prioritization of all transactions on PuffChain. If there is a conflict, only the transaction with the highest priority is kept, and the nodes discard the rest of the lower-priority transactions. This approach efficiently and effectively handles conflicting transactions and improves overall consensus efficiency.

C. Change of Proposers

Resisting Transaction Censorship: The original PBFT protocol [19] maintains the proposer unchanged unless a failure triggers the view change. This design allows the proposer to have the authority to determine which transactions should be processed, resulting in transaction censorship [25]. This lack of fairness can be particularly problematic in a permissionless environment. To address this issue, PuffChain implements mandatory proposer changes. Upon receipt of a committed block, a new round begins with a new proposer, thus distributing transaction processing responsibility across participants.

Ensuring System Liveness: In PBFT, an unresponsive proposer can halt the consensus protocol. To prevent indefinite node waiting and ensure liveness, PBFT uses a view-change mechanism. Similarly, in PuffChain, a Proposer-Change message triggers the view change mechanism. If over two-thirds of the committee members consider the current proposer faulty,

TABLE I
BFT CONSENSUS PROTOCOL IN FOUR CASES

Case	case ₁	case ₂	case ₃	case ₄
Proposer	$\mathcal{H}$	$\mathcal{H}$	$\mathcal{A}$	$\mathcal{A}$
Committee	$\mathcal{H}$	$\mathcal{A}$	$\mathcal{H}$	$\mathcal{A}$
Probability	$p_P^{\mathcal{H}}p_C^{\mathcal{H}}$	$p_P^{\mathcal{H}}p_C^{\mathcal{A}}$	$p_P^{\mathcal{A}}p_C^{\mathcal{H}}$	$p_P^{\mathcal{A}}p_C^{\mathcal{A}}$

proposer replacement is initiated. The validator with the highest ID value becomes the next proposer, maintaining system reliability and continuity.

V. THROUGHPUT ANALYSIS AND MAXIMIZATION

Different protocol parameters can lead to varying blockchain throughput, even when the underlying network and consensus protocol remain the same. For example, the Nakamoto consensus protocol would yield different throughput if the block size is not 1 MB or the block interval is not 10 minutes, even if the underlying network is fixed and there are enough transactions in the network. Therefore, it is critical to explore how the parameters of a blockchain system influence its performance.

In this section, we first introduce a theoretical model to analyze the throughput of PuffChain. Then, we derive the optimal parameters for the number and size of data blocks to achieve the maximum blockchain throughput.

PBFT guarantees blockchain safety if the number of faulty nodes is less than $\frac{1}{3}$ of the number of committee members. Thus, if the proportion of votes held by adversarial nodes in the committee is less than $\frac{1}{3}$ of the total votes, the committee is honest, and transactions appended to the blockchain can be accepted by all honest nodes. According to whether the proposer and the committee are malicious or honest, the BFT consensus protocol can be divided into four cases, as shown in Table I.

The role sortition is random, and the probability of a node being selected is proportional to its weight. Thus, the probability that the proposer is an adversarial node $p_P^{\mathcal{A}}$ is equivalent to the proportion of the weight of adversarial nodes. That is,

$$p_P^{\mathcal{A}} = \beta. \quad (8)$$

Correspondingly, the probability of an honest proposer $p_P^{\mathcal{H}}$ is

$$p_P^{\mathcal{H}} = 1 - \beta. \quad (9)$$

If the total number of votes of the committee is $\mathcal{T}$, the probability of an honest committee $p_C^{\mathcal{H}}$ is equivalent to that the number of votes held by adversarial nodes in the committee is less than $\mathcal{T}/3$, that is

$$p_C^{\mathcal{H}} = \sum_{i=0}^{\lfloor \mathcal{T}/3 \rfloor} \binom{\mathcal{T}}{i} \beta^i (1 - \beta)^{\mathcal{T}-i}. \quad (10)$$

A committee is malicious when adversarial nodes hold more than $\frac{1}{3}$ of the total votes. The probability of a malicious committee $p_C^{\mathcal{A}}$ is equivalent to that the number of votes held by the

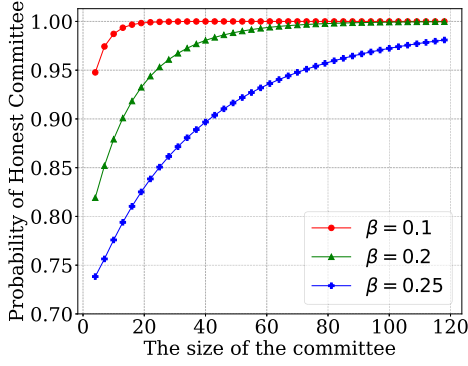


Fig. 3. Confidence of the honest committee p_C^H .

honest nodes is less than $\frac{2}{3}$, that is:

$$p_C^A = \sum_{i=0}^{\lceil 2T/3 \rceil} \binom{T}{i} (1-\beta)^i \beta^{T-i}. \quad (11)$$

As shown in Fig. 3, the larger the committee, the less likely adversarial nodes hold more than $\frac{1}{3}$ of the total votes. When the committee is large enough, the probability of an honest committee approaches 1, and the probability of adversarial nodes holding more than $\frac{1}{3}$ of the total votes is negligible. Thus, the probability of case 2 and case 4 in Table I is negligible. We only consider case 1 and case 3.

Although in case 3 the malicious proposer may select valid data blocks, we only consider the worst-case blockchain throughput, where all data blocks selected by a malicious proposer are invalid. Suppose each control block points to n data blocks ($n \in \mathbb{N}$), and each data block contains m transactions ($m \in \mathbb{N}$), and the average transaction size is $\bar{x}$. The expected throughput of valid transactions is:

$$\Gamma = \frac{mnp_P^H p_C^H \bar{x}}{t}, \quad (12)$$

where t is the total latency of reaching one round of consensus.

Our three-phase BFT consensus protocol involves seven kinds of messages, combined with network assumptions, we can get

$$\begin{aligned} t &= \frac{h \|role\|}{b} + d + \frac{h \|B_C\|}{b} + d + \frac{h \|R_B\|}{b} + d + \frac{h \|B_D\|}{b} + d \\ &\quad + \frac{h \|M_P\|}{b} + d + \frac{h \|M_C\|}{b} + d + \frac{h \|M_R\|}{b} + d \\ &= \frac{h}{b} (\|role\| + \|B_C\| + \|R_B\| + \|B_D\| + \|M_P\| + \|M_C\| \\ &\quad + \|M_R\|) + 7d, \end{aligned} \quad (13)$$

where $\|role\|$, $\|B_C\|$, $\|R_B\|$, $\|B_D\|$, $\|M_P\|$, $\|M_C\|$, $\|M_R\|$ are the size of role, control block, request for data block, data block, *Validate*, *Commit* and *Reply* message, respectively.

Each node starts a timer when it enters a new round. If the expiration time that we set for one round is $t_{\max}$, then $t \leq t_{\max}$. $t_{\max}$ is determined by analyzing network latency, node processing rate, and real-world performance data.

Thus, the problem of maximizing the worst-case throughput of PuffChain can be formulated as follows:

$$\underset{m,n}{\text{maximize}} \quad \Gamma \quad (14a)$$

$$\begin{aligned} \text{s.t.} \quad t &= \frac{h}{b} (\|role\| + \|B_C\| + \|R_B\| + \|B_D\| \\ &\quad + \|M_P\| + \|M_C\| + \|M_R\|) + 7d, \end{aligned} \quad (14b)$$

$$t \leq t_{\max}, \quad (14c)$$

$$m, n \in \mathbb{N}. \quad (14d)$$

$\|B_C\|$, $\|M_P\|$, $\|M_C\|$, and $\|M_R\|$ are linearly related to the number of selected data blocks n , and $\|B_D\|$ is linearly related to the number of transactions m . The size of the data block ID and the size of the transaction ID are both a ($a > 0$). Thus, we can re-denote them as

$$\begin{aligned} \|B_C\| &= an + \theta(\|B_C\|), \|B_D\| = am + m\bar{x} + \theta(\|B_D\|), \\ \|M_P\| &= an + \theta(\|M_P\|), \|M_C\| = an + \theta(\|M_C\|), \\ \|M_R\| &= an + \theta(\|M_R\|), \end{aligned} \quad (15)$$

where $\theta(\cdot)$ represents constants independent of m and n .

Let $g = \|role\| + \|R_B\| + \theta(\|B_C\|) + \theta(\|B_D\|) + \theta(\|M_P\|) + \theta(\|M_C\|) + \theta(\|M_R\|)$, so Ineq. (14b) can be simplified to

$$\begin{aligned} t &= \frac{h}{b} [4an + am + m\bar{x} + \|role\| + \|R_B\| + \theta(\|B_C\|) \\ &\quad + \theta(\|B_D\|) + \theta(\|M_P\|) + \theta(\|M_C\|) + \theta(\|M_R\|)] + 7d \\ &= \frac{h}{b} (4an + am + m\bar{x}) + \frac{h}{b} g + 7d. \end{aligned} \quad (16)$$

Equation (16) can be transformed into:

$$4an + am + m\bar{x} = \frac{b}{h} \left(t - \frac{h}{b} g - 7d \right). \quad (17)$$

According to Cauchy-Schwarz inequality, combined with (17), we can get

$$\sqrt{4a(a+\bar{x})} \sqrt{mn} \leq \frac{4an+m(a+\bar{x})}{2} = \frac{b}{2h} \left(t - \frac{h}{b} g - 7d \right). \quad (18)$$

In (18) can be transformed into:

$$mn \leq \frac{(4an+m(a+\bar{x}))^2}{16a(a+\bar{x})} = \frac{b^2}{16h^2a(a+\bar{x})} \left(t - \frac{h}{b} g - 7d \right)^2. \quad (19)$$

Compared with the objective function (14a), we transform Ineq. (19) as follows:

$$\begin{aligned} \Gamma &= \frac{mnp_P^H p_C^H \bar{x}}{t} \leq \frac{(4an + m(a + \bar{x}))^2 p_P^H p_C^H \bar{x}}{16a(a + \bar{x})t} \\ &= \frac{b^2 p_P^H p_C^H \bar{x}}{16h^2a(a + \bar{x})t} \left(t - \frac{h}{b} g - 7d \right)^2, \end{aligned} \quad (20)$$

with the equality holding if and only if $4an = m(a + \bar{x})$ according to Cauchy-Schwarz inequality. Thus, the maximum value of Γ is equal to the maximum value of $f(t)$, where

$$f(t) = \frac{b^2 p_P^H p_C^H \bar{x}}{16h^2a(a + \bar{x})t} \left(t - \frac{h}{b} g - 7d \right)^2. \quad (21)$$

When $t \geq \frac{h}{b}g + 7d$, $\frac{\partial f(t)}{\partial t} \geq 0$, $f(t)$ is monotonically increasing. According to (16) and (14c), we can get

$$\frac{h}{b}g + 7d \leq t \leq t_{\max}. \quad (22)$$

Thus, when $t = t_{\max}$, $f(t)$ can take the maximum value:

$$\max_t f(t) = \frac{b^2 p_P^H p_C^H \bar{x}}{16 h^2 a(a + \bar{x}) t_{\max}} \left(t_{\max} - \frac{h}{b}g - 7d \right)^2. \quad (23)$$

Therefore, combining with (20), (21) and (23), when $t = t_{\max}$ and $4an = m(a + \bar{x})$, we can get

$$\begin{aligned} n &= \frac{b}{8ah} \left(t_{\max} - \frac{h}{b}g - 7d \right), \\ m &= \frac{b}{2(a + \bar{x})h} \left(t_{\max} - \frac{h}{b}g - 7d \right), \end{aligned} \quad (24)$$

$$\max_{m,n} \Gamma = \max_t f(t) = \frac{p_P^H p_C^H \bar{x} b^2 \left(t_{\max} - \frac{h}{b}g - 7d \right)^2}{16 a(a + \bar{x}) h^2 t_{\max}}. \quad (25)$$

VI. SECURITY ANALYSIS

A. Committee Security

We start by establishing the resistance of the role selection algorithm to Sybil attacks.

Theorem 1 (Resistance to Sybil Attacks): Our role sortition algorithm ensures that the expected number of stake units of a node being selected is proportional to its stakes.

Proof: When $u = 1$, the left side of I^u is:

$$\sum_{k=0}^0 P_{w_i}^k = \sum_{k=0}^0 \binom{w_i}{k} p^k (1-p)^{w_i-k} = (1-p)^{w_i}. \quad (26)$$

When $u = w_i$, the right side of I^u is:

$$\sum_{k=0}^{w_i} P_{w_i}^k = \sum_{k=0}^{w_i} \binom{w_i}{k} p^k (1-p)^{w_i-k} = 1. \quad (27)$$

Combining with (26) and (27), we can get $\cup_u I^u = [0, 1]$. Thus, the sortition algorithm divides the interval $[0, 1]$ into consecutive intervals.

Let p denote the probability of a stake unit being selected. Assume a node with stake w and ψ . Let X be the random variable that represents the number of stake units that are selected by the sortition algorithm. According to Section III-B1, X takes on the value u when the random number ψ is in the interval I^u . Therefore, the probability that $X = u$ is equal to the length of the interval I^u , which is:

$$\begin{aligned} P(\psi \in I^u) &= \sum_{k=0}^u \binom{w}{k} p^k (1-p)^{w-k} - \sum_{k=0}^{u-1} \binom{w}{k} p^k (1-p)^{w-k} \\ &= \binom{w}{u} p^u (1-p)^{w-u} = P(X = u). \end{aligned} \quad (28)$$

According to (28), X follows a binomial distribution with parameters w and p , i.e. $X \sim B(w, p)$. The expected value of X is $E(X) = wp$.

Assume two nodes i and j with stake weights w_i and w_j , respectively. Let X_i and X_j be the random variables that represent the number of stake units that are selected by the sortition algorithm for nodes i and j , respectively. Then, the expected number of selected units for each of them satisfies the following relationship:

$$\frac{E(X_i)}{E(X_j)} = \frac{w_i p}{w_j p} = \frac{w_i}{w_j}. \quad (29)$$

Therefore, adversarial nodes cannot launch Sybil attacks by splitting their stake w_i into multiple parts since the expected number of selected units for a node is proportional to its stake. $\square$

The proof demonstrates that splitting stakes among multiple IDs does not increase the influence of an adversarial node in the selection process, providing resistance against Sybil attacks.

Next, we analyze the security of committee formation under the role sortition algorithm. The proportion of adversarial nodes' weight to the total weight of the system is β , and β does not exceed $\frac{1}{3}$. To ensure the security of a committee, it is necessary to ensure that in a committee, the number of votes held by adversarial nodes does not exceed $\frac{1}{3}$ of the total number of votes of the committee.

Theorem 2 (Committee Security): For each round that employs the role sortition algorithm, given $\beta < \frac{1}{3}$ and a sufficiently large total number of votes in the committee $\mathcal{T}$, PuffChain ensures that the committee is honest with overwhelming probability.

Proof: The committee security can be modeled as a random sampling problem that results in malicious committees p_C^A or honest committees. The probability of a malicious committee is as follows:

$$p_C^A = \Pr \left[\mathcal{H} \leq \frac{2}{3} \mathcal{T} \right] = \sum_{i=0}^{\lceil 2\mathcal{T}/3 \rceil} \binom{\mathcal{T}}{i} (1-\beta)^i \beta^{\mathcal{T}-i}. \quad (30)$$

According to (30), p_C^A decreases exponentially as $\mathcal{T}$ increases. Therefore, when $\beta < \frac{1}{3}$ and the total number of votes in the committee $\mathcal{T}$ is sufficiently large, PuffChain ensures that the committee is honest with overwhelming probability. $\square$

Thus, the theorem establishes that as long as the number of adversarial nodes remains below a certain threshold, the committee remains secure and robust.

B. Safety Analysis

Because our BFT consensus protocol is built on PBFT, the security of the protocol also strongly depends on PBFT. We now provide an analysis of the safety properties of PuffChain.

Lemma 1 (Uniqueness of the: Validate Whitelist) Given two *Validate* messages accepted by honest nodes in the same round, their associated whitelists are identical.

Proof: Proof by contradiction. Assume there are two *Validate* messages $Validate_1 = \langle Validate, r, id_{j_1}, H(B_C^r), L_{j_1}^r \rangle_{\sigma_{j_1}}$ and $Validate_2 = \langle Validate, r, id_{j_2}, H(B_C^r), L_{j_2}^r \rangle_{\sigma_{j_2}}$ in round r , where $L_{j_1}^r \neq L_{j_2}^r$, but both are accepted by honest nodes. Thus, $L_{j_1}^r$ receives more than $\frac{2}{3} \mathcal{T}$ valid votes, and $L_{j_2}^r$ receives

more than $\frac{2}{3}\mathcal{T}$ valid votes. The total number of votes for $L_{j_1}^r$ and $L_{j_2}^r$ in the *Validate* phase of round r is more than $2 \cdot \frac{2}{3}\mathcal{T} = \frac{4}{3}\mathcal{T}$.

Note that any honest node cannot vote more than once in a phase in the same round, but any adversarial node may maliciously vote on two messages. Thus, the total number of votes for two different *Validate* whitelists $L_{j_1}^r$ and $L_{j_2}^r$ in round r does not exceed the sum of the number of honest nodes' votes and twice the number of the adversarial nodes' votes, which is less than $\frac{4}{3}\mathcal{T}$. Therefore, there is a contradiction with the assumption. Honest nodes cannot accept two different *Validate* whitelists in the same round. The *Validate* whitelist accepted by honest nodes in each round is unique, $L_{j_1}^r = L_{j_2}^r$.

According to the Lemma 1, the *Validate* whitelist accepted by honest nodes in a round is the same. Similarly, only *Commit* whitelist with more than $\frac{2}{3}\mathcal{T}$ votes can be accepted by honest nodes. The *Commit* whitelist accepted by honest nodes in a round is the same. Thus, the whitelists in the *Reply* messages generated by honest nodes are the same.

Because the number of votes held by adversarial nodes in the committee is less than $\frac{1}{3}\mathcal{T}$, adversarial nodes cannot forge a whitelist in *Reply* message that has received more than $\frac{1}{3}\mathcal{T}$ votes. If a participating node i receives a whitelist with more than $\frac{1}{3}\mathcal{T}$ votes in *Reply* messages, the whitelist must have been accepted by at least one honest node in the committee in *Commit* phase. Thus, other honest nodes in the committee have reached a consensus on the whitelist in the round. The committed control block and data blocks that the participating node i appends to the blockchain in the round are the same as the committed control blocks and data blocks appended to the blockchain by honest nodes in the committee. According to the synchronous network assumption, any honest participating node will receive *Reply* messages within a bounded time. Any honest node will receive *Reply* messages from the honest nodes in the committee and append the same control block and data blocks as the honest nodes in the committee. Therefore, the committed control block and data blocks appended by all honest nodes in this round are the same. PuffChain achieves safety with $\beta < \frac{1}{3}$ in the synchronous network.

C. Liveness Analysis

Similar to PBFT, our protocol ensures liveness using a proposer change mechanism. If committee members are unable to reach a consensus for a long time, their timers will expire, and the proposer change mechanism will be triggered. According to the proposer-change mechanism, a new proposer will continue the consensus. Thus, the proposer change mechanism prevents nodes from waiting indefinitely if the BFT consensus protocol halts for a long time. In addition, it is impossible to stop consensus by frequently broadcasting *Proposer-Change* messages. The change occurs only if the next proposer receives more than $\frac{2}{3}\mathcal{T}$ votes, but adversarial nodes hold no more than $\frac{1}{3}\mathcal{T}$ votes.

D. Resistance to Various Attacks

PuffChain incorporates multifaceted defenses across its architecture and protocols to resist many attacks in blockchain

systems. First, the selection probability strictly depends solely on the ground-truth stake possessed, irrespective of any fabricated identities. Theorem 1 proves PuffChain against Sybil attacks. Second, the transaction allocation based on packer identities, combined with the embedded validation filters within the three-phase BFT consensus, collectively provides a robust framework mitigating double-spending risks. Third, the selection probability is based on historical, immutable stake weights from ν rounds prior, as elaborated in (28), preventing manipulation. The role sortition contains embedded proofs bounding the manipulability of adversaries attempting stake grinding. In addition, PuffChain's dynamic scaling abilities indirectly enhance security by preventing exploitable overload scenarios. Finally, the use of BFT consensus enhances resistance against Denial of Service, and three-role decoupling mitigates transaction censorship attacks. Through these security constructions, PuffChain reliably resists various attacks, ensuring system integrity despite significant adversarial challenges.

VII. PERFORMANCE EVALUATION

We have implemented PuffChain in Go and deployed it on Amazon's EC2 c2.large instances for evaluation. Each instance hosts a PuffChain node with 2 vCPU cores, 4 GB of RAM, and a 10 Gbps network interface.

By default, each node connects to 8 random peers. Generally, a total of 100 PuffChain nodes are connected to form the peer-to-peer network. The network diameter is 4 hops. To simulate a realistic network latency between cities, we add a propagation delay of 120 ms on each link. The upload and download rate of each link is 10 Mbps. The size of each transaction is set to 512 Bytes.

A. Network Performance

1) *Network Latency*: Our network model assumes that network latency increases linearly with message size in Section II-B. First, we use a source to send messages ranging in size from 1 MB to 4 MB, recording the time it takes for 99%, 75%, and 50% nodes in the network to receive the message. Then, we use different numbers of sources (1,10,20,30,40,60,80,100) to send messages with size 1 MB, recording the time it takes for 99%, 75%, and 50% nodes in the network to receive the messages. The results are shown in Fig. 4(a) and Fig. 4(b), respectively.

As shown in Fig. 4, when there is only one source, the network latency increases almost linearly with the size of the message. When the message size is 1 MB, the network latency increases almost linearly with the number of sources in the network.

To examine the effect of decoupling functionalities on network latency, we fix the amount of data but change the number of sources and measure the network latency for 99% of the nodes in the network to receive this data. As shown in Fig. 8, we conduct three groups of comparative experiments with data sizes of 2 MB, 3 MB, and 4 MB, respectively. They are 1) one source sending 2 MB messages and two sources sending different 1 MB messages, respectively; 2) one source sending 3 MB messages and three sources sending different 1 MB

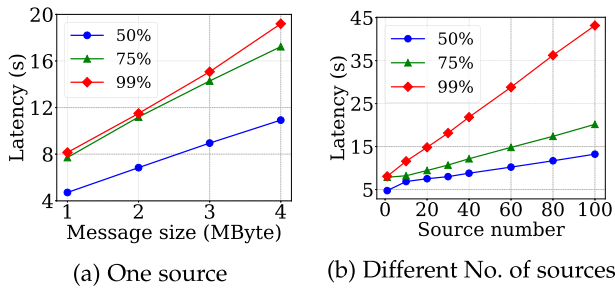


Fig. 4. Network latency with (a) one source sending messages of different sizes and (b) the different numbers of sources sending messages with Msgsize=1 MB.

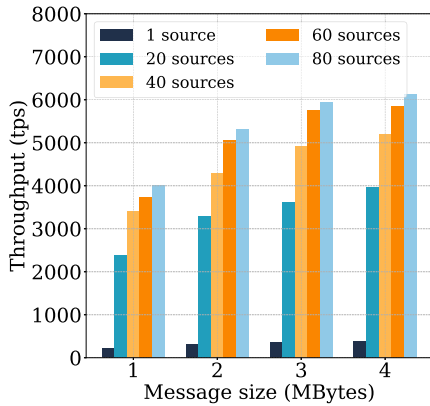


Fig. 5. Network throughput with different message sizes and number of sources.

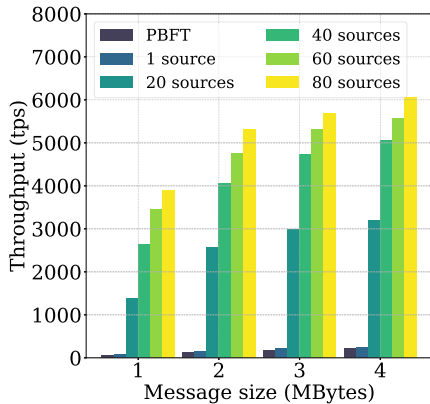


Fig. 6. Blockchain throughput with different block sizes and different numbers of packers.

messages, respectively; 3) one source sends 4 MB messages and four sources to send different 1 MB size messages, respectively. In addition, we also measure the network latency for sending 1 MB messages from one source as a control.

Fig. 8 illustrates that distributing the data across multiple sources for parallel broadcasting substantially reduces network latency compared to a single source transmission. When there is only one source, the network latency increases significantly as the number of data increases. If multiple sources are used to

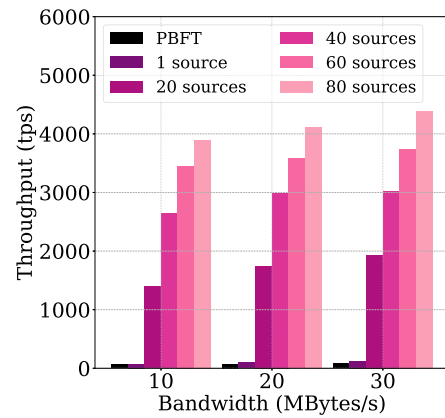


Fig. 7. Blockchain throughput with different bandwidth and different number of packers.

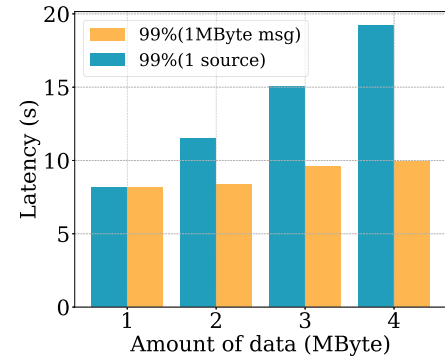


Fig. 8. Comparison of network latency with the same amount of data sent by: 1) one source, 2) multiple sources with each Msgsize =1 MB.

send messages in parallel, the network latency increases slightly as the amount of data increases.

2) *Network Throughput*: PuffChain aims to make the maximum blockchain throughput close to the network's capacity. We define network capacity as the rate of transaction processing, specifically, the number of transactions per second (tps) successfully delivered to 99% of the nodes in the network. We investigate this capacity under various conditions, examining the impact of different numbers of sources (1, 20, 40, 60, and 80) and message sizes (1 MB, 2 MB, 3 MB, and 4 MB), as depicted in Fig. 5.

The following two observations can be made from Fig. 5. First, with a constant message size, an increase in the number of sources correlates with a higher network throughput. However, beyond a certain point, the incremental gain in throughput becomes marginal as the number of sources continues to rise. Second, when the number of sources remains constant, enlarging the message size contributes to an improvement in network throughput. The rate of increase in network throughput diminishes as the message size grows larger. The observations indicate that within certain limits, both an increase in the number of sources and message size can positively influence network throughput. However, as the number of sources increases or

message size increases, the network becomes more congested, and the increase in network throughput becomes slight. Notably, when 80 sources each transmit 4 MB messages, the network achieves a throughput of 6130 tps, suggesting a close approximation to the network's capacity under these conditions.

B. Blockchain Performance

This subsection assesses PuffChain's performance by examining its scalability with varying factors like data block size, the number of packers, network bandwidth, and committee size. PuffChain's performance metrics are compared to those of the original PBFT protocol, serving as a baseline for our analysis.

1) *The Effect of Data Blocks*: We measure the effect of the size of data blocks and the number of data blocks per round (i.e., the number of packers) on the throughput of PuffChain. For comparison, the original PBFT protocol is implemented without a packer. This setup provides a baseline against which PuffChain's enhancements can be measured. The upload and download rates are 10 Mbps per link. Fig. 6 presents the PuffChain throughput with different data block size (1 MB, 2 MB, 3 MB, and 4 MB) and the number of packers per round (0, 1, 20, 40, 60, and 80).

From Fig. 6, the following observations can be made: First, with only one packer, PuffChain demonstrates a slight improvement in throughput compared to the baseline PBFT protocol. Second, when the block size remains constant, blockchain throughput increases with the number of packers, but the increase becomes less significant as more packers are added. Third, with a steady number of packers, larger block sizes result in higher throughput, but the growth in throughput becomes less pronounced with increasing block sizes. These observations suggest that increasing the number of packers or the size of data blocks within a certain range can improve PuffChain throughput. Beyond this range, the network becomes more congested, and the PuffChain throughput increases slightly.

Comparing the network and blockchain evaluation results, we can make the following observations:

- 1) Trends of increasing the number of sources/packers or the size of messages/data blocks similarly affect network and PuffChain throughput.
- 2) When there are 80 sources/packers, and the message/data block size is 4 MB, the maximum PuffChain throughput of 6061 tps approaches the network throughput of 6130 tps.

2) *The Effect of Network Bandwidth*: PuffChain's throughput is impacted by network latency, which is influenced by the bandwidth. According to the network model (1), the larger the bandwidth, the smaller the network latency. We varied the bandwidth (10 Mbps, 20 Mbps, 30 Mbps) to evaluate the effect of bandwidth on PuffChain throughput when the data block size is 1 MB. As demonstrated in Fig. 7, enhancing the bandwidth can improve blockchain throughput, but the effect is relatively limited.

3) *The Effect of Committee Size*: For BFT-based PoS blockchains, the committee size is a critical factor affecting both blockchain security and consensus efficiency. When the network size is 100 nodes, and the committee size is less than 60, we find that the committee size has a negligible effect on

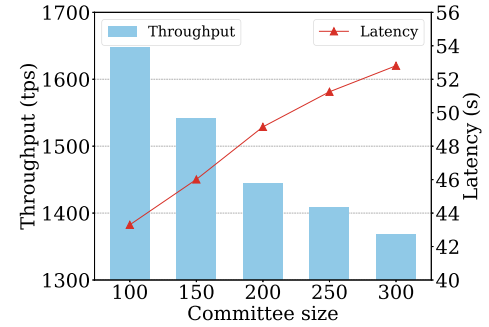


Fig. 9. Effect of committee size on blockchain throughput and latency.

TABLE II
COMPARISON OF EXISTING BLOCKCHAINS

Scheme	Consensus	De-duplicate Tx	Adaptive Throughput
Conflux [9]	PoW	N	N
Prism [10]	PoW	N	N
Nezha [26]	PoW	Y	N
BCUBE [2]	PoS&PoW	N	N
GeckoDAG [27]	PoW	Y	N
Adaptchain [6]	PoW	Y	Y
PuffChain	PoS	Y	Y

The bold content is the work introduced in this paper.

the blockchain throughput and block confirmation latency. To further evaluate the effect of committee size on blockchain throughput and latency, we expand the network to 400 nodes and varied the committee size from 100 to 300. The other parameters are kept constant: 40 packers, a block size of 1 MB, and a network bandwidth of 10 Mbps. The results, as depicted in Fig. 9, indicate a decline in throughput and an increase in latency when the committee size exceeds 100.

C. Comparing PuffChain With Other Blockchains

This section provides a qualitative comparison of PuffChain against both influential solutions such as Conflux [9] and Prism [10], as well as emerging blockchain protocols including AdaptChain [6] and Nezha [26]. As quantitative benchmarks across diverse platforms and codebases remain infeasible, we focus on contrasting the distinct functional capabilities of these systems. Table II summarizes the key comparative attributes.

Compared to existing blockchain systems, PuffChain offers three unique advantages.

1) *Sustainable consensus protocol*: PuffChain adopts a PoS consensus, significantly improving energy efficiency and reducing environmental impact, contrasting with the PoW schemes of other systems such as AdaptChain [6]. This shift lowers electricity consumption and minimizes hardware costs, positioning PuffChain as a more sustainable blockchain system.

2) *Transaction de-duplication to maximize effective throughput*: PuffChain de-duplicates transactions using transaction allocation, avoiding propagation and storage of duplicated transactions. In contrast, other solutions, such as Conflux [9] and Prism [10], report an overall throughput, which includes a considerable number of duplicated and conflicting transactions.

3) *Adaptive throughput scaling to match demand*: By adjusting the number of packers, PuffChain can increase or decrease throughput to match the user transaction demand. Other solutions, such as Nezha [26] and GeckoDAG [27], are limited by static, pre-configured parameters, reducing their adaptability to fluctuating transactions. This distinguishes PuffChain as the only PoS-based blockchain with adaptive throughput.

In summary, PuffChain's sustainable consensus, adaptive throughput scaling, and transaction de-duplication collectively distinguish it as a superior blockchain protocol compared to other decentralized solutions.

VIII. RELATED WORK

PoW blockchains have explored decoupling functionalities as a scaling approach. Bitcoin-NG [28] serves as a pioneer by decoupling the functionalities of miners into leader election and transaction serialization. A leader can generate several microblocks containing transactions without solving puzzles until a new leader solves the next puzzle. Although Bitcoin-NG improves throughput, it is vulnerable to censorship attacks and DoS attacks [24]. Byzcoin [25] introduces two parallel blockchains for leader election and transaction verification. Prism [10] further decouples the functionalities into block proposal, validation, and confirmation. One proposal block points to multiple transaction blocks to increase throughput, and multiple voting blocks point to one proposal block to reduce confirmation latency. In addition, there are many studies [6], [26], [27] improve blockchain scalability by allowing parallel generation of blocks, but they are PoW based, which is a waste of energy.

PoS blockchains have emerged as an energy-efficient alternative to PoW blockchains. Chain-based PoS blockchains, such as the Ouroboros series [8], [29], [30], [31], have forks and require chain selection rules to ensure blockchain consistency. BFT-based PoS blockchains, such as Tendermint [32], involve randomly selecting nodes to form committees and execute classic BFT consensus algorithms. In Algorand [7], nodes run VRF locally to know whether they are selected to vote for improved Byzantine agreement protocol. Recently, BCUBE [2] utilizes a unique Byzantine broadcast protocol named OVERLAYBB for pipelined block generation, ensuring high throughput even in malicious majority conditions. Compared with these PoS schemes, PuffChain can dynamically adjust its throughput according to transaction demand.

Sharding-based blockchains improve throughput by multiple committees' parallel processing. Elastico [15] distributes nodes to different committees according to the least significant bits of PoW hash. Omniledger [16] argues that committee formation in Elastico is not strongly bias-resistant because nodes can selectively discard PoW hash. OmniLedger uses RandHound [33] to generate randomness to sample representative validators. A variant of Byzcoin [25] is used to process transactions within each shard. After that, RapidChain [17] is built on Information Dispersal Algorithms (IDA) [34] to accelerate message broadcasting. Dang et al. [35] argue that RapidChain does not achieve isolation. It is worth noting that all sharding-based blockchains face the challenge of handling cross-shard transactions [36].

These systems allocate transactions randomly [37], leading to many cross-shard transactions, which can lower throughput and double confirmation times. However, such issues are not present in PuffChain, which employs a different approach to processing transactions.

IX. CONCLUSION

PuffChain is a dynamically scaling blockchain system that can achieve optimal effective throughput. PuffChain achieves dynamic scalability by adjusting the number of packers based on user transaction demand. The three-phase consensus protocol with filters ensures efficient block validation and enhances system resource utilization. We have also developed a theoretical model to analyze the system's throughput and derive optimal protocol parameters. This ensures that PuffChain performs at its peak, maximizing its effectiveness. Extensive experiments have been implemented on Amazon EC2 clusters to evaluate the performance of PuffChain. Our results have shown that PuffChain can achieve 6061 transactions per second with 100 nodes.

ACKNOWLEDGMENT

The authors gratefully acknowledge the financial support from the under Grant CityU 11213920, Grant CityU 11218521, Grant R1012-21, Grant R6021-20F, Grant RFS2122-1S04, Grant C1029-22G, and Grant C2004-21G.

REFERENCES

- [1] S. Nakamoto, "BITCOIN: A peer-to-peer misc cash system." 2008. Accessed: Feb. 14, 2024. [Online]. Available: <http://www.bitcoin.org/bitcoin.pdf>
- [2] R. Hou, H. Yu, and P. Saxena, "Using throughput-centric byzantine broadcast to tolerate malicious majority in blockchains," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 1263–1280.
- [3] R. Zhang, D. Zhang, Q. Wang, S. Wu, J. Xie, and B. Preneel, "NC-Max: Breaking the security-performance tradeoff in Nakamoto consensus," in *Proc. 29th Netw. Distrib. System Secur. Symp.*, 2022, pp. 1–18.
- [4] C. Xu, C. Zhang, J. Xu, and J. Pei, "SlimChain: Scaling blockchain transactions through off-chain storage and parallel processing," *Proc. VLDB Endowment*, vol. 14, no. 11, pp. 2314–2326, 2021.
- [5] X. Luo, Z. Xu, K. Xue, Q. Jiang, R. Li, and D. Wei, "ScalaCert: Scalability-oriented PKI with redactable consortium blockchain enabled" on-cert" certificate revocation," in *Proc. IEEE 42nd Int. Conf. Distrib. Comput. Syst.*, 2022, pp. 1236–1246.
- [6] J. Xu, Q. Xie, S. Peng, C. Wang, and X. Jia, "AdaptChain: Adaptive scaling blockchain with transaction deduplication," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 6, pp. 1909–1922, Jun. 2023.
- [7] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, and N. Zeldovich, "Algorand: Scaling byzantine agreements for cryptocurrencies," in *Proc. 26th Symp. Operating Syst. Princ.*, 2017, pp. 51–68.
- [8] T. Kerber, A. Kiayias, M. Kohlweiss, and V. Zikas, "Ouroboros cryptosynous: Privacy-preserving proof-of-stake," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 157–174.
- [9] C. Li et al., "A decentralized blockchain with high throughput and fast confirmation," in *Proc. USENIX Annu. Tech. Conf. Assoc.*, 2020, pp. 515–528.
- [10] V. Bagaria, S. Kannan, D. Tse, G. Fanti, and P. Viswanath, "PRISM: Deconstructing the blockchain to approach physical limits," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 585–602.
- [11] J. Xu, C. Wang, and X. Jia, "A survey of blockchain consensus protocols," *ACM Comput. Surv.*, vol. 55, no. 13, pp. 1–35, 2023.
- [12] S. Liu et al., "Flexible advancement in asynchronous BFT consensus," in *Proc. 29th Symp. Operating Syst. Princ.*, 2023, pp. 264–280.

- [13] V. Buterin and V. Griffith, "Casper the friendly finality gadget," *CoRR*, vol. abs/1710.09437, 2017. [Online]. Available: <http://arxiv.org/abs/1710.09437>
- [14] Y. Lu, Z. Lu, and Q. Tang, "BOLT-DUMBO transformer: Asynchronous consensus as fast as the pipelined BFT," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2159–2173.
- [15] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A secure sharding protocol for open blockchains," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 17–30.
- [16] E. Kokoris-Kogias, P. Jovanovic, L. Gasser, N. Gailly, E. Syta, and B. Ford, "OmniLedger: A secure, scale-out, decentralized ledger via sharding," in *Proc. IEEE Symp. Secur. Privacy*, 2018, pp. 583–598.
- [17] M. Zamani, M. Movahedi, and M. Raykova, "Rapidchain: Scaling blockchain via full sharding," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 931–948.
- [18] S. Deb, S. Kannan, and D. Tse, "POSAT: Proof-of-work availability and unpredictability, without the work," in *Proc. 25th Int. Conf. Financial Cryptography Data Secur., ser. Lecture Notes Comput. Sci.*, 2021, vol. 12675, pp. 104–128.
- [19] M. Castro and B. Liskov, "Practical byzantine fault tolerance," in *Proc. 3rd USENIX Symp. Operating Syst. Des. Implementation*, 1999, pp. 173–186.
- [20] S. Micali, M. Rabin, and S. Vadhan, "Verifiable random functions," in *Proc. 40th Annu. Symp. Foundations Comput. Sci. 40th Annu. Symp. Foundations Comput. Sci.*, 1999, pp. 120–130.
- [21] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham, "HOT-STUFF: BFT consensus with linearity and responsiveness," in *Proc. ACM Symp. Princ. Distrib. Comput.*, 2019, pp. 347–356.
- [22] Y. Sompolinsky, Y. Lewenberg, and A. Zohar, "SPECTRE: A fast and scalable cryptocurrency protocol," *Cryptol. ePrint Arch.*, Report 2016/1159, 2016, <https://ia.cr/2016/1159>
- [23] X. Li, Y. Zheng, K. Xia, T. Sun, and J. Beyler, "Phantom: An efficient privacy protocol using ZK-snarks based on smart contracts," *Cryptol. ePrint Arch.*, Report 2020/156, 2020, <https://ia.cr/2020/156>
- [24] H. Yu, I. Nikolić, R. Hou, and P. Saxena, "OHIE: Blockchain scaling made simple," in *Proc. IEEE Symp. Secur. Privacy*, 2020, pp. 279–296.
- [25] E. Kokoris-Kogias, P. Jovanovic, N. Gailly, I. Khoffi, L. Gasser, and B. Ford, "Enhancing bitcoin security and performance with strong consistency via collective signing," in *Proc. 25th USENIX Conf. Secur. Symp.*, 2016, pp. 279–296.
- [26] J. Xiao et al., "Nezha: Exploiting concurrency for transaction processing in dag-based blockchains," in *Proc. IEEE 42nd Int. Conf. Distrib. Comput. Syst.*, 2022, pp. 269–279.
- [27] X. Dai et al., "GeckoDAG: Towards a lightweight DAG-Based blockchain via reducing data redundancy," in *Proc. IEEE 43rd Int. Conf. Distrib. Comput. Syst.*, 2023, pp. 451–462.
- [28] I. Eyal, A. E. Gencer, E. G. Sirer, and R. Van Renesse, "Bitcoin-NG: A scalable blockchain protocol," in *Proc. 13th USENIX Symp. Netw. Syst. Des. Implementation*, 2016, pp. 45–59.
- [29] A. Kiayias, A. Russell, B. David, and R. Oliynykov, "Ouroboros: A provably secure proof-of-stake blockchain protocol," in *Proc. 37th Annu. Int. Cryptology Conf. Lecture Notes Comput. Sci.*, 2017, vol. 10401, pp. 357–388.
- [30] B. David, P. Gaži, A. Kiayias, and A. Russell, "Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2018, pp. 66–98.
- [31] C. Badertscher, P. Gaži, A. Kiayias, A. Russell, and V. Zikas, "Ouroboros genesis: Composable proof-of-stake blockchains with dynamic availability," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2018, pp. 913–930.
- [32] J. Kwon, "Tendermint: Consensus without mining," *Draft v. 0.6, Fall*, vol. 1, no. 11, pp. 1–11, 2014.
- [33] E. Syta et al., "Scalable bias-resistant distributed randomness," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 444–460.
- [34] N. Alon, H. Kaplan, M. Krivelevich, D. Malkhi, and J. Stern, "Scalable secure storage when half the system is faulty," in *International Colloquium on Automata, Languages, and Programming*. Berlin, Germany: Springer, 2000, pp. 576–587.
- [35] H. Dang, T. T. A. Dinh, D. Loghin, E.-C. Chang, Q. Lin, and B. C. Ooi, "Towards scaling blockchain systems via sharding," in *Proc. Int. Conf. Manage. Data*, 2019, pp. 123–140.
- [36] Z. Hong, S. Guo, and P. Li, "Scaling blockchain via layered sharding," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 12, pp. 3575–3588, Dec. 2022.
- [37] H. Huang et al., "Brokerchain: A cross-shard blockchain protocol for account/balance-based state sharding," in *Proc. IEEE Conf. Comput. Commun.*, 2022, pp. 1968–1977.



Jie Xu received the B.Eng. degree from the Department of Information Security, University of Science and Technology of China (USTC), Hefei, China, in 2017, and the M.Eng. degree from the Department of Electronic Engineering and Information Science, USTC in 2020. She is currently working toward the Ph.D. degree with the Department of Computer Science, City University of Hong Kong, Hong Kong. Her research interests include network security and applied cryptography.



Sen Peng received the B.E. degree in computer science and technology from the Southern University of Science and Technology, Shenzhen, China, in 2020. He is currently working toward the Ph.D. degree with the Department of Computer Science, City University of Hong Kong, Hong Kong. His research interests include artificial intelligence security and cloud security.



Cong Wang (Fellow, IEEE) is currently a Professor with the Department of Computer Science, City University of Hong Kong, Hong Kong. His research interests include data and network security, blockchain and decentralized applications, and privacy-enhancing technologies. Mr. Wang was the corecipient of the IEEE INFOCOM Test of Time Paper Award 2020, the Best Paper Award of IEEE ICDCS 2020, ICPADS 2018 and MSN 2015, the Best Student Paper Award of IEEE ICDCS 2017, the Outstanding Researcher Award (2019), the Outstanding Supervisor Award (2017), and the President's Awards (2016 and 2019). He is a Founding Member of the Young Academy of Sciences of Hong Kong, Hong Kong and a Research Fellow of the Hong Kong Research Grants Council. He is the Editor-in-Chief of IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING. He is a Fellow of IEEE.



Xiaohua Jia (Fellow, IEEE) received the B.Sc. and M.Eng. degrees from the University of Science and Technology of China, Hefei, China, in 1984 and 1986, respectively, and the D.Sc. degree in information science from the University of Tokyo, Tokyo, Japan, in 1991. He is currently a Chair Professor with the Department of Computer Science, City University of Hong Kong, Hong Kong. His research interests include data privacy and security, cloud computing, and distributed systems. He is an Editor of IEEE TRANSACTIONS ON COMPUTERS, *Journal of Networks*, and *World Wide Web: Internet and Web Information Systems*. He is the General Chair of IEEE ICDCS 2023, ACM ICN 2019, and ACM MobiHoc 2008. He is a Fellow of IEEE.